\# AI Programming Foundations Project: Module Summary

\## Overview

This project builds a reproducible data workflow using public Cincinnati Fire/EMS CAD incident data. The purpose of the workflow is to practice the full early-stage data process: loading a real CSV file, inspecting structure and quality, cleaning selected fields, creating useful analysis features, producing exploratory summaries, and communicating findings through visualizations and interpretation. The project does not perform machine learning or operational decision support. It is limited to academic data workflow practice.

\## Dataset Description

The dataset used in this project is `cincinnati\_fire\_incidents\_2025.csv`, a public Cincinnati Fire/EMS CAD incident dataset. The local file contains 97,881 rows and 17 columns. The fields include incident location information, incident creation time, dispatch time, arrival time, closure time, incident type, disposition, beat, neighborhood, and community council neighborhood.

This dataset was selected because it is public, tabular, large enough for meaningful exploration, and closely related to the type of municipal incident data that may be useful in later public-safety analytics projects. The dataset does not include patient data, departmental data, or PHI. Because the file is a real municipal export, it also includes realistic data-quality issues such as missing values, text fields that need standardization, and timestamp columns stored as text.

\## Workflow Description

The workflow begins by importing pandas, NumPy, and Matplotlib. The CSV file is then loaded into a pandas DataFrame. The notebook displays the dataset shape, column names, data types, sample rows, descriptive statistics, and missing-value counts.

The first cleaning function, `clean\_text\_columns`, strips whitespace from text columns and standardizes blank text values as missing values. This supports cleaner grouping and counting during exploratory analysis. The second cleaning function, `convert\_time\_columns`, converts the incident time fields from text into datetime values. This step is important because the original CSV stores timestamp values as strings, which limits time-based analysis until conversion is complete.

After cleaning, the notebook creates three analysis features:

1\. `incident\_hour`

2\. `incident\_day\_name`

3\. `dispatch\_to\_arrival\_minutes`

The `dispatch\_to\_arrival\_minutes` field is calculated from primary unit dispatch and arrival timestamps. This makes it possible to explore one simplified response-time interval, while still recognizing that this interval is not a complete measure of operational performance.

The notebook also includes an exploratory data analysis function, `summarize\_incident\_data`, which prints the dataset shape, top incident types, top dispositions, top neighborhoods, and summary statistics for dispatch-to-arrival time.

\## Key Decisions and Assumptions

One key decision was to keep the original public CSV file in the repository because the file size is manageable and it helps reviewers rerun the notebook without needing to download the data separately. Another decision was to preserve the original column names rather than renaming every field. This keeps the notebook connected to the source export while still allowing selected derived fields to be added.

The workflow uses `INCIDENT\_TYPE\_ID` as the main incident-type field because `INCIDENT\_TYPE\_DESC`, `CFD\_INCIDENT\_TYPE`, and `CFD\_INCIDENT\_TYPE\_GROUP` contain substantial missing values in this local file. The notebook therefore avoids over-relying on columns that are mostly empty. This is a practical data-quality decision rather than a claim that those fields are unimportant.

The response-time calculation assumes that `DISPATCH\_TIME\_PRIMARY\_UNIT` and `ARRIVAL\_TIME\_PRIMARY\_UNIT` represent comparable timestamps for the primary responding unit. Missing, invalid, negative, or extreme intervals are handled cautiously during visualization by filtering the histogram to values between 0 and 60 minutes.

Reproducible workflow design is important because reviewers and future users need to rerun the analysis and understand each step. Danchev (2022) emphasizes reproducible data science practices using Python-based workflows, which supports this project’s use of a structured notebook, local data file, and requirements file. Wickham (2014) also supports the importance of organizing and cleaning tabular data so that it can be manipulated, visualized, and interpreted more reliably.

\## Results and Interpretation

The dataset contains 97,881 incident records and 17 columns. Initial inspection showed that most core fields were populated, including location, agency, creation time, event number, and coordinates. Some fields had notable missing values. For example, `INCIDENT\_TYPE\_DESC`, `CFD\_INCIDENT\_TYPE`, and `CFD\_INCIDENT\_TYPE\_GROUP` were missing in most records. This guided the decision to use `INCIDENT\_TYPE\_ID` for incident-category exploration.

The top incident type IDs included EMS-related and fire-alarm-related categories. The most common incident type ID was `EMS`, followed by `=FALARM`, `PERDWN - 32D1 UNKNOWN`, `ACCI - (C) =`, and `=INFOF`. This suggests that the dataset includes a mix of EMS, fire alarm, accident, information, and other response categories.

The top dispositions included transport, investigation, cancellation, refusal, transfer, and release outcomes. The most common disposition was `TRL: TRANSPORT - LIGHTS/SIREN`. Other common dispositions included `IN: INVESTIGATION`, `CNOS: CANCELLED - ON SCENE/GOA`, `PRF: PATIENT REFUSED EVAL/CARE`, and `PTX: PATIENT TX - TRANSFER EMS`.

Neighborhood counts showed that incident volume was not evenly distributed across the city. The highest-count neighborhoods included Westwood, Downtown, Avondale, East Price Hill, West Price Hill, Over-the-Rhine, West End, Walnut Hills, CUF, and College Hill. These results show how neighborhood grouping can support later geographic or community-level analysis, while still requiring caution because incident counts alone do not account for population, call density, staffing, hazards, or reporting practices.

Figure 1 shows the top 10 incident type IDs. This chart makes the most common call categories easier to compare than a raw frequency table. Figure 2 shows incident counts by hour of day, which supports basic time-pattern exploration. Figure 3 shows the distribution of dispatch-to-arrival times after filtering to values between 0 and 60 minutes. The filtering decision makes the visualization easier to interpret by reducing the influence of extreme or invalid values.

\## Responsible Practice

This dataset is public municipal incident data, but responsible use still matters. Incident records may reflect community conditions, reporting practices, dispatch coding, resource availability, and system documentation habits. High incident volume in a neighborhood should not be interpreted as a simple statement about the people who live there. It may reflect many overlapping factors, including population density, service demand, geography, infrastructure, and public reporting patterns.

The workflow also avoids making operational, clinical, or performance claims from the data. Dispatch-to-arrival time is only one simplified interval and does not capture the full incident timeline, resource constraints, travel conditions, call severity, staging, scene safety, or patient outcome. Because of that, the notebook treats response-time exploration as a data workflow exercise rather than a performance evaluation.

A major data-quality limitation is missingness. Several descriptive incident-type fields are mostly missing in this local file. This could affect interpretation if those fields were expected to contain more detailed classification information. The workflow responds by documenting missing values and using more complete fields for analysis.

\## Reproducibility

The project is designed so the notebook can be rerun from top to bottom. The repository includes the local CSV dataset, the main notebook, a README with run instructions, and a `requirements.txt` file generated from the Python environment. Git is used to track project progress, including separate commits for the README, notebook, and requirements file. The repository also includes a branch beyond `main`, which supports the required Git workflow.

The notebook was executed successfully using Jupyter through Python. The executed notebook includes code outputs and visualizations, which helps connect the written interpretation to actual results produced by the workflow.

\## Sources and Citations

City of Cincinnati. (n.d.). *Cincinnati Fire Incidents (CAD) including EMS: ALS/BLS*. Cincinnati Open Data. https://data.cincinnati-oh.gov/Safety/Cincinnati-Fire-Incidents-CAD-including-EMS-ALS-BL/vnsz-a3wp

Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education, 5*(56), 156. https://doi.org/10.21105/jose.00156

Wickham, H. (2014). Tidy data. *Journal of Statistical Software, 59*(10), 1–23. https://doi.org/10.18637/jss.v059.i10